第 2 章推導解答

DS02-01 • CSR 與轉置

CSR 是 `row_ptr=[0,1,3,4]`、`col_idx=[1,0,3,1]`、`values=[5,-2,7,4]`。原欄計數 `[1,2,0,1]`，起點 `[0,1,3,3]`。依原項順序，寫入位置依序為 1、0、3、2。轉置輸出為 `(0,1,-2),(1,0,5),(1,2,4),(3,1,7)`，尺寸 4×3。再以新欄計數 `[1,2,1]` 和起點 `[0,1,3]` 轉回，得到原尺寸與原排序三元組。驗證必須比對尺寸及座標值，不只比 nnz。

DS02-02 • 四維位址與容量

線性偏移為 $(((i_0D_1+i_1)D_2+i_2)D_3+i_3)$，展開為 $i_0D_1D_2D_3+i_1D_2D_3+i_2D_3+i_3$。位元組偏移乘 w。每個 $0\le i_j<D_j$，尺寸乘積與再乘 w 都必須能以配置 API 的尺寸型別表示，且配置需求不能超過容器或平台上限。索引合法仍不足以排除中間乘法溢位，檢查須先於乘法。

DS02-03 • KMP 追蹤

$\pi = [0,1,2,3,4,0]$。以下 q 表示已匹配的 pattern 長度，文字索引從 0 起算；每列處理同一個當前字元直到匹配或退到 $q=0$，不倒退文字索引。

文字索引 i	T[i]	讀入前 q	失配回退	讀入後 q
0	a	0	無	1
1	a	1	無	2
2	a	2	無	3
3	a	3	無	4
4	a	4	無	5
5	a	5	P[5]=b 失配，q=π[4]=4；同一個 a 再匹配 P[4]	5
6	a	5	5→4，再匹配 P[4]	5
7	a	5	5→4，再匹配 P[4]	5
8	a	5	5→4，再匹配 P[4]	5
9	b	5	與 P[5]=b 匹配，不回退	6

在 $i=9$ 得 $q=6=|P|$，第一個匹配起點為 $9-6+1=4$。直觀演算法會從起點 0、1、2、3 各重新比較五個 a 才在 b 的位置失配，從起點 4 再比較六字元成功，共 30 次字元比較。對一般同型長文字，重複工作可達 $\Theta(NM)$；KMP 每次回退降低 q，所有增加又由前進字元支付，加上建立 π 的工作共 $O(N+M)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 2 章推導解答 （續）

DS02-04 • CSR 是否一定省空間

COO 每項兩個索引加一個值，共 $16k$。CSR 每項欄號和值共 $12k$，再加 $4(R+1)$。CSR 較省 iff $12k+4(R+1)<16k$，即 $R+1<k$。指定數據下 COO 約 1,600 bytes，CSR 約 4,001,204 bytes，列指標主導。CSR 可能仍便於逐列運算，但不能聲稱空間一定較小。此比較排除標頭、對齊、壓縮與不同索引寬度，不能當成所有實作的精確配置量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 2 章推導解答 （續）

原創驗算程式附錄: transpose.cpp

```
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <stdexcept>
#include <tuple>
#include <vector>

struct Entry {
	std::size_t row;
	std::size_t col;
	int value;
	bool operator==(const Entry& other) const {
		return std::tie(row, col, value) == std::tie(other.row, other.col,
other.value);
	}
};

struct Sparse {
	std::size_t rows;
	std::size_t cols;
	std::vector<Entry> entries;
};

void validate(const Sparse& matrix) {
	for (std::size_t i = 0; i < matrix.entries.size(); ++i) {
		const auto& item = matrix.entries[i];
		if (item.row >= matrix.rows || item.col >= matrix.cols || item.value == 0) {
			throw std::invalid_argument("invalid sparse entry");
		}
		if (i != 0) {
			const auto& before = matrix.entries[i - 1];
			if (std::tie(before.row, before.col) >= std::tie(item.row, item.col)) {
				throw std::invalid_argument("entries must be sorted and unique");
			}
		}
	}
}

Sparse transpose(const Sparse& matrix) {
	validate(matrix);
	std::vector<std::size_t> count(matrix.cols, 0);
	for (const auto& item : matrix.entries) {
```

第 2 章推導解答 （續）

```
                ++count[item.col];
        }
        std::vector<std::size_t> next(matrix.cols, 0);
        for (std::size_t col = 1; col < matrix.cols; ++col) {
                next[col] = next[col - 1] + count[col - 1];
        }
        Sparse result{matrix.cols, matrix.rows, std::vector<Entry>(matrix.entries.size())};
        for (const auto& item : matrix.entries) {
                result.entries[next[item.col]++] = {item.col, item.row, item.value};
        }
        return result;
}

void require(bool condition, const char* message) {
        if (!condition) {
                throw std::runtime_error(message);
        }
}

int main() {
        const Sparse matrix{3, 4, {{0, 1, 5}, {1, 0, -2}, {1, 3, 7}, {2, 1, 4}}};
        const auto result = transpose(matrix);
        const std::vector<Entry> expected{{0, 1, -2}, {1, 0, 5}, {1, 2, 4}, {3, 1, 7}};
        require(result.rows == 4 && result.cols == 3 && result.entries == expected,
                "transpose does not match slide example");
        const auto restored = transpose(result);
        require(restored.rows == matrix.rows && restored.cols == matrix.cols &&
                restored.entries == matrix.entries, "double transpose changed matrix");
        const auto empty = transpose(Sparse{0, 4, {}});
        require(empty.rows == 4 && empty.cols == 0 && empty.entries.empty(), "empty dimensions
lost");
        bool rejected = false;
        try {
                transpose(Sparse{2, 2, {{1, 0, 7}, {0, 0, 2}}});
        } catch (const std::invalid_argument&) {
                rejected = true;
        }
        require(rejected, "unsorted input was accepted");
        std::cout << "PASS transpose: exact COO, double transpose, empty, invalid input\n";
}
```

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study